

10 Chapter 10: Sequential Circuit Design

10.1 Introduction

To reiterate, gate delay t_{pd} refers to the total propagation delay within a CMOS device; that is, the delay between the input signal reaching the device's gate, and the output value switching to the intended value (this intended value depends on noise margins and the measurement parameters).

t_{pd} itself depends on several variables, which include the characteristics of the circuit and the devices themselves (covered in Chapter 4), extrinsic factors such as the transition times of the associated input signals and the capacitive loads on the output (covered in Chapters 6 & 9), and operating conditions such as temperature, process variation, and supply voltage (covered in Chapter 7).

When modeling gate delay, the reality is that there is no one value of t_{pd} that applies to an entire logic gate; instead, propagation delay varies as a function of the direction of the switching (high-to-low or low-to-high) as well as the associated inputs. Timing arc diagrams are one way to easily display this information. Often, singular logic gates are interconnected into a combinational logic circuit (covered in Chapter 9) with multiple possible paths for the relevant signal. In these cases, an important area of analysis is deriving the minimum and maximum path delay of a given combinational logic circuit. The most naive approach, of course, is to evaluate every path from input to output and determine which is the *critical setup path*; that is, the one which produces the maximum delay (the minimum delay may be found using the same method, and finding the *critical hold path*). However, as our delay model's complexity (and accuracy) increases, this methodology becomes increasingly unwieldy when accounting for factors such as asymmetric rise and fall delay, interconnect-related delay, as well as the effects of transition time and output loading. Logical effort is a convenient way to represent delay associated with individual logic gates when doing critical path analysis in a combinational circuit.

Moving on from the category of combinational logic circuits, this chapter will cover *sequential* logic circuits, which combine logic gates with memory components, the most common of which are latches and flip-flops; however, with memory elements come timing concerns. In digital systems, there must be some way of identifying *when* to read a bit, not just identify whether it represents a 1 or 0. To do this, we would normally sample the value being sought at a constant time interval; the interval in which we can capture the bit under consideration is known as the "eye", and so variations in the circuit's timing elements will, naturally, shorten this valid sampling period t_E .

In short, one of the major concerns in the design and analysis of sequential logic circuits is when to sample bits and how to decide whether a bit is "valid" (an acceptable voltage that clearly represents a low signal or a high signal). In this discussion, for the purposes of generality, we'll refer to the values being captured and read as "tokens". Due to the virtually unavoidable variability in timing and delay that we've just discussed, it's often necessary to delay tokens that arrive too early, to enforce tight sequencing and timing requirements. On the flip side, however, these elements add an element of delay known as *sequencing overhead*; on the bright side, though, sequencing overhead is a known quantity (within reason) that can be used to characterize the circuit's behavior more accurately than can be done with unknown variables.

10.2 Sequencing Static Circuits

Sequential circuits themselves generally include a clock along with some memory element(s); in our case, latches and flip-flops.

Beginning with the clock, usually implemented in modern technology using a crystal oscillator, it generates a rising and falling edge at a consistent interval, thus providing the circuit's memory elements with a sort of timing "anchor" that keeps the millions of elements in a modern-day CMOS circuit synchronized. Obviously, though, this is the ideal case. Getting the clock edge to every part of the circuit at the same time is not truly feasible in practice. Generally, an input clock will fan out to a number of repeaters, and these will then fan out to twice as many repeaters, and so on, until all registers are covered. Keeping interconnect length

standardized will also help, and so an H-tree layout is a common approach.

Clocks may be defined in terms of various properties, including their period, t_{per} , frequency, $f = \frac{1}{t_{\text{per}}}$, pulse width, t_H and t_L , respectively, for the high and low pulses, duty cycle, t_H/t_{per} , and rise and fall time, t_R and t_F , respectively. The total period consists of the high and low pulse widths as well as the rise and fall times,

$$t_{\text{per}} = t_H + t_F + t_L + t_R.$$

Moving on to sequencing elements, latches and flip-flops are the two primary devices used, and flops are normally edge-triggered (level-triggered devices may also be used, but are less reliable). Latches are generally level sensitive, and remain transparent (that is, the input value D is transferred directly to the output Q) as long as the clock is high. When the clock is low, though, the latch holds the last value of Q that was present prior to the clock going low. Flops, on the other hand, trigger at the rising or falling edge of the clock, copying the value of D to Q , and ignoring D at all other times.

The operation of a latch is represented using a timing diagram. Open/closed (opaque) state. Propagation delays are important factors in D-latch operation (as with any gate), notated as $t_{\text{PLH}}(\text{GQ})$, etc. Timing parameters for a D-latch include the setup and hold time (**what does setup and hold mean?**), as well as the minimum pulse width of the high G state. D input should target the overlap between the opening and closing of G. Datasheet example of a D-latch, gives parameters we just talked about. Some setup and hold values are negative, meaning that setup rising high can occur *before* the clock opening the gate, and the data will still be captured.

We can put two latches back to back, with output and input buffers, to form a D flip-flop. Explanation of how this kind of dynamic flip-flop works. In this kind of back-to-back flip-flop, the data goes through two "layers" of flip-flop to ensure no data is lost? (**check this functionality**). This is an **edge-triggered** flip-flop.

D flip-flops have a specific timing diagram and parameters, similar to a latch. Setup and hold, in this case, defines the window for capturing data, as usual (either setup or hold can be negative, but their sum must be greater than zero). This allows us to derive max delay for a flip flop as

$$t_{pd} \leq T_c - (t_{\text{setup}} + t_{pcq}) \text{ **check the derivation of this.**}$$

Sum of the delays around the flip flop loop (with feedback/buffer) is a function of the clock frequency, the setup time, and the Elmore delay, so that gives us the minimum clock frequency (**I think?**)

Max clock frequency is limited by the critical (longest) path in the device. Quiz will include min/max clock frequencies dependent on critical paths through a circuit.

Enable is a common addition to a flip-flop that turns it into a feedback loop; you may also gate the clock to enable/disable the clock. Many flip-flops don't need to change that often in, say, a register file, so we can save power by disabling the clock when not needed. Reset is another common feature that sets output low, usually used for starting up and getting a known state, can be synchronous or asynchronous.

There are also min/max delay requirements/specs for flip-flops, has to do with hold time violations.

Looks at slides/example of hold timing path. When is hold constraint met/not met? More importantly, how can we fix this? Sequencing methods. Max delay derivation **check on this too.**

What is time borrowing?

Clocks are not ideal, unfortunately; this leads to clock skew! The clock arrives at different times in different places, unfortunately, and a clock also has a rise and fall time, that is likely asymmetric. Clock jitter, also bad! Caused by noise. Statistical analysis of jitter?

10.3 Metastability & Synchronization

Back to D flip-flops. In summary, at the rising edge of the clock, the D-input is held in a feedback loop and output to the Q-output, until the next rising edge of the clock. The D input must hold a *stable* 0 or 1 during the rising edge of the clock, defined by the setup and hold times. There is max time t_{CQ} in which the Q output must take on the value of the D input to operate correctly, by definition.

In synchronization, it becomes more difficult to determine the sequence of two events the closer they occur to each other in time. Closer the race, the harder to call. When the order of events is not possible to resolve, the synchronizer will enter a **metastable** state, and may remain there indefinitely. While this is happening, the output may not be correct.

A simple example would consist of a NAND arbiter (or S-R latch). This is a way of *arbitrating* between two possible values. In one case, let's say A and B arrive at the same time (the difference is less than the gate delay). Due to the physics of the device, this generates a voltage difference ΔV , which is then amplified (William Dally textbook). If the voltage eventually reaches V_{DD} , the output will stabilize over a period known as the decision time, t_d . If $\Delta t = 0$, t_d grows to infinity! In practice, this does not occur, because even small amounts of electromagnetic noise will cause the circuit to stabilize in one state or another.

This concept extends to latches and flip-flops as well, which can become similarly trapped in a metastable state for an indeterminate amount of time. When do things go metastable, how do we deal with it?